DATA301 Progress Report - Sam Ladbrook

LSH Accelerated Collaborative Filtering for Movie Recommendations

What's Been Done:

Proposal and Planning:

- Submitted the project proposal covering the research question, algorithm design, dataset selection and a four week timetable.
- Read MMDS Chapters 3 and 9 covering MinHash, Locality Sensitive Hashing and item-based collaborative filtering in detail to refresh myself.
- Read through related sections of three foundation papers directly relevant to my project algorithms: Linden et al. (2003) on item-to-item collaborative filtering, Adomavicius & Tuzhilin (2005) on recommender systems, and Broder (1997) on MinHash resemblance — the mathematical foundation for the signature implementation. Although reasonably old papers they are very insightful and still relevant.

Data Loading and preprocessing (Week 1):

- Downloaded the Amazon Movies and TV 5-core dataset directly into Google Colab using wget.
- Loaded the raw gzip compressed JSON into a Dash Bag and parsed each record to extract the key three fields: userId, itemId and rating.
- Converted the Dask Bag to a Dask DataFrame and computed dataset statistics which confirmed there are 36,537 unique items after filtering to items with at least 5 positive ratings. (Ratings of 3 stars and over)
- Performed an 80/20 train/test split by user, randomly holding out 20% of each user's ratings as the test set and saved outputs as train.csv and test.csv for session persistence.

MinHash implementation (Week 2):

- Built item-to-user sets using a groupby operation on positive ratings, producing a dictionary mapping each item to a set of users who rated it positively.
- Implemented MinHash signature generation from scratch using numpy broadcasting. Each hash function takes the form h(x) = (ax + b) mod p, with randomly generated parameters per function and the minimum hash value across all users in the set is retained.
- Computed 100-function MinHash signatures for all 36,537 items in 3.7 seconds.
- Validated MinHash accuracy against true Jaccard similarity. For the most similar item pair in the dataset (sharing 15 users), the true Jaccard similarity was **0.028** and the MinHash estimate was **0.020**; a difference of only **0.008**, confirming the implementation is working correctly.

Seeing the seemingly low similarity values made me reconsider whether my research question was still relevant. However the low Jaccard similarity values are expected in sparse datasets like the Amazon Movies and TV one, where user overlap between items is minimal. Despite this, the MinHash estimates closely approximate the true similarities, showing that LSH-based methods can preserve similarity relationships between items. Therefore the result remains valid and relevant.

LSH banding and pair generation (Week 2):

- Implemented LSH banding, splitting each 100 value signatures into 50 bands of 2 rows and grouping items into candidate buckets where any band matches.
- Generated **837,178** candidate pairs from a possible **838,266,985** brute-force pairs; a **99.9%** reduction in comparisons, done in 12.9 seconds.

Parameter tuning and project direction change:

- I started with LSH parameters of 20 bands x 5 rows. This was too aggressive for the sparse dataset. This produced only **476** candidate pairs, insufficient for meaningful recommendations.

- I then decided to adjust the parameters to 50 bands x 2 rows and lowered the positive rating threshold that was at 4 stars to 3 stars. This fixed the issue and produced **837,178** candidate pairs reported above.
- The parameter sensitivity will be discussed as a limitation in the written report.

Although the initial LSH parameters created a potential issue for the project by producing too few candidate pairs, I was able to resolve this through parameter tuning, so no major change in project direction was required.

Summary of key results so far:

Metric	Result
Unique items (5+ positive ratings)	36,537
MinHash signature computation time (100 functions)	3.7 seconds
True Jaccard similarity	0.028 (error: 0.008)
LSH candidate pairs generated	837,178
Brute-force comparison count	838,266,985
LSH reduction in comparisons	99.9%
LSH banding computation time	12.9 seconds

What's Next:

The following steps are still to be done to ensure I complete the project on time.

Week 3 (May 17-23) - Collaborative filtering implementation:

- Implement cosine similarity computation within LSH candidate buckets to produce item-item similarity scores for the accelerated pipeline.
- Implement the brute-force alternative cosine similarity across all item pairs as the baseline for comparison.

- Generate top 10 movie recommendations for test users from both the LSH-accelerated and brute-force approaches by aggregating similarity weighted ratings from each user's training history.
- Run both pipelines with three MinHash configurations (50, 100, 200) to measure how accuracy changes with signature length.

Week 4 (May 24-30) - Evaluation and write up:

- Evaluate both approaches against held-out test ratings using Precision@10, RMSE and Mean Absolute Error (MAE). MAE will be included as an additional metric as I think it will be more interpretable for the 1-5 star rating scale.
- Record and compare runtime for both approaches to quantify the speed benefit of LSH.
- Produce a results table comparing the accuracy and runtime across the configurations.
- Write the final report covering the test results, critique of design and project reflection.
- Clean up and fully document final code file for submission.

Road blocks:

Currently there are no significant road blocks preventing progress at this stage. The main technical challenge encountered was LSH parameter tuning for the sparse dataset; the initial banding configuration produced too few candidate pairs to be useful as discussed earlier. However I managed to identify this and resolve the issue by adjusting the band and row parameters and lowering the positive rating threshold and is now documented as a known limitation of the approach rather than an ongoing problem. Google Colab session resets are a minor practical consideration, but this has been mitigated by saving the train/test splits to CSV files so preprocessing does not need to be repeated. No issues have been encountered with the Dask implementation of dataset access.